SOCIAL MEDIA
SHORT-FORM VIDEO TRACKER

Product requirements • UX specification • Technical architecture • Build roadmap

Document	Specification for a cross-platform browser extension
Working product name	ScrollMeter (working name; rename as desired)
Platforms	Facebook Reels, Instagram Reels, TikTok, YouTube Shorts, and future supported short-video feeds
Primary goal	Count watched short-form videos, identify creators where possible, show recent activity, and send user-scheduled summaries
Target build	Desktop Chrome / Edge extension using Manifest V3; other browsers after compatibility testing

Important scope note: This is a cross-platform short-form video tracker, not a Facebook-only tracker. Each social platform requires its own detection adapter. Platform page structures and access rules change, so detection is best-effort and must be tested and maintained.

Prepared 1 November 2026 • Version 2.0

1. Product overview

Product vision: Build a simple, privacy-conscious browser extension that helps people understand how much short-form video content they consume across supported social media websites.

The extension should work quietly in the background while the user browses supported websites. It detects short-form videos that are actively viewed, records a viewing event, groups activity by platform and creator, and displays clear statistics in a minimal popup dashboard.

Supported platforms — planned scope

Platform	Content type	MVP support approach
YouTube	Shorts	Detect Shorts watch pages and visible Shorts feed items; distinguish Shorts from ordinary videos.
Instagram	Reels	Detect Reels pages/feed items where the page exposes reliable identifiers and creator details.
Facebook	Reels	Detect Reels pages/feed items; avoid counting unrelated video posts.
TikTok	Short videos	Detect active video items and creator identity where visible.
Future platforms	Other short-video feeds	Add a separate adapter only after feasibility, policy, and reliability review.

Do not promise that every platform, page layout, embedded video, or mobile app can be tracked. This specification targets browser pages on desktop. It does not automatically monitor native mobile apps.

2. Main user features

• **Cross-platform tracking:** show activity separately for each supported platform and combined across platforms.

• **Reel / Short counter:** count unique short-form videos that meet the view threshold.

• **Creator tracking:** show creator/channel/profile name and per-creator counts when those details can be identified.

• **Recent activity:** list recently viewed items with platform, creator, time, and a link to the original post when available.

• **Time tracking:** estimate watch duration only when reliable playback/visibility measurements are available.

• **Daily and weekly analytics:** total videos, unique creators, platform breakdown, estimated time, and trends.

• **Scheduled notifications:** let users choose a local notification time and receive a daily report.

• **Privacy controls:** pause tracking, disable individual platforms, clear history, export data, and choose retention.

3. Minimal UI and interaction

The UI should feel lightweight and neutral. Avoid a crowded analytics dashboard. Prioritise the daily total and the platform breakdown.

Popup layout

Area	Contents
Header	Product mark/name; small settings icon; tracking status indicator.
Today's summary	Total short videos watched today; estimated watch time; unique creators.
Platform breakdown	Small rows for YouTube, Instagram, Facebook, TikTok with counts.
Recent activity	Latest 5–10 items with creator, platform icon, and time.
Navigation	Overview • Creators • History • Settings.
Settings	Tracking toggles per platform; daily report toggle; notification time; privacy controls.

Optional in-page indicator

If an in-page indicator is included, make it small, dismissible, and disabled by default or user-controlled. Example: “Shorts today: 18 · All platforms: 42”. Never cover playback controls, captions, navigation, or creator information.

Use original visual design. Do not copy another extension's branding or imply affiliation with Meta, Google, TikTok, or other platforms.

4. Counting rules and accuracy

A video element appearing in the page is not enough to count a view. Pages often preload several videos, autoplay previews, or keep off-screen videos mounted in the DOM.

Recommended initial view rule: count a short-form item only when it is the active item, sufficiently visible in the viewport, and playing or otherwise demonstrably being viewed for a configurable threshold (for example, 2 seconds). De-duplicate by a stable content identifier.

Event data model

```
{
  "eventId": "generated-unique-id",
  "platform": "youtube",
  "contentType": "short",
  "contentId": "platform-specific-id-if-available",
  "contentUrl": "canonical-url-if-available",
  "creatorId": "creator-id-if-available",
  "creatorName": "Creator name or Unknown creator",
  "startedAt": 1793520000000,
  "watchSeconds": 8,
  "detectionConfidence": "high"
}
```

Use `platform + contentId` as the preferred de-duplication key. If the platform does not expose a stable ID, use a carefully normalised canonical URL. Do not guess a creator. Save “Unknown creator” where necessary and expose an optional correction mechanism.

Clearly label estimated metrics. A “viewed item” count is not the same as the platform's official view count. Do not claim exact watch time if the browser cannot measure actual playback reliably.

5. Technical architecture

Use a shared core with one adapter per platform. This is the most important architectural decision: do not build one large script full of platform-specific selectors.

```
Supported social website
        |
        v
Platform content script / adapter
        |
        v
Normalised viewing event
        |
        v
Background service worker
        |
        +--> Local event store
        +--> Daily / weekly aggregation
        +--> Scheduled notification
        |
        v
Popup dashboard / settings
```

Module	Responsibility
manifest.json	Manifest V3 configuration, permissions, supported site matches, popup, service worker.
platform adapters	Separate detection logic for YouTube Shorts, Instagram Reels, Facebook Reels, TikTok.
event normaliser	Convert platform-specific data into one shared event format.
background.js	Validate messages, de-duplicate events, store data, schedule notifications.
storage layer	Local-first event storage and settings; later consider IndexedDB for larger history.

Module	Responsibility
analytics.js	Daily totals, weekly totals, creator ranking, platform breakdown.
popup / options UI	Overview, creators, history, notification settings, privacy controls.

Recommended technology

Layer	Recommended choice
Extension platform	Chrome Extension Manifest V3
Languages	JavaScript, HTML, CSS; TypeScript optional for larger codebase
UI	Vanilla HTML/CSS/JS for MVP, or React + Vite if dashboard complexity grows
Browser storage	chrome.storage.local for settings/small MVP; IndexedDB for larger event history
Scheduling	chrome.alarms
Notifications	chrome.notifications
Testing	Manual browser tests plus unit tests for normalisation and aggregation

6. Platform adapter design

Each adapter must answer the same questions: Is this a supported short-form video page/item? Which item is active? What is its stable ID or canonical URL? Who is the creator, if visible? Is it currently visible and playing? When should a view event be emitted?

```
// Shared adapter contract (illustrative)
const adapter = {
  platform: "youtube",
  matchesLocation(location) {
    // Return true only for the platform routes this adapter supports.
  },
  findActiveItems() {
    // Return candidate short-form items currently visible.
  },
  extractMetadata(item) {
    // Return contentId, contentUrl, creatorName, creatorUrl.
  },
  isActivelyViewed(item) {
    // Check visibility, active state and playback signals.
  }
};
```

Platform-specific cautions

• **YouTube Shorts:** distinguish Shorts from standard YouTube videos; handle feed navigation that may update without a full page load.

• **Instagram Reels:** handle login-gated content, changing routes, and creator labels that may be absent from some feed cards.

• **Facebook Reels:** avoid confusing ordinary Watch videos or video posts with Reels.

• **TikTok:** handle single-video pages and scrolling feeds; do not treat every embedded video as a TikTok feed view.

Use public page UI signals only where permitted. Do not bypass authentication, private endpoints, anti-bot controls, paywalls, or platform access restrictions. Before distributing the extension, review the current terms and developer/store policies of each platform.

7. Suggested project folder structure

```
social-short-tracker/
  manifest.json
  background/
    service-worker.js
    alarms.js
    notifications.js
  content/
    shared/
      visibility.js
      event-normaliser.js
      dedupe.js
    adapters/
      youtube-shorts.js
      instagram-reels.js
      facebook-reels.js
      tiktok.js
  storage/
    events.js
    settings.js
  analytics/
    daily-summary.js
    creator-stats.js
    platform-stats.js
  popup/
    popup.html
    popup.css
    popup.js
  options/
    options.html
    options.css
    options.js
  icons/
    icon16.png
    icon48.png
    icon128.png
  tests/
    dedupe.test.js
    analytics.test.js
```

This structure is a recommended organisation, not a requirement to create every file on day one. Start with one platform and the shared event pipeline, then add adapters.

8. Permissions and manifest planning

Request the minimum permissions required. Avoid `<all_urls>` if the product only supports a known list of social domains. The exact host patterns should match the platforms actually implemented and be explained in the store listing.

```
{
  "manifest_version": 3,
  "name": "ScrollMeter - Short Video Tracker",
  "version": "1.0.0",
  "description": "Understand your short-form video viewing across supported websites.",
  "permissions": [
    "storage",
    "alarms",
    "notifications"
  ],
  "host_permissions": [
    "https://www.youtube.com/*",
    "https://www.instagram.com/*",
    "https://www.facebook.com/*",
    "https://www.tiktok.com/*"
  ],
  "background": {
    "service_worker": "background/service-worker.js"
  },
  "action": {
    "default_popup": "popup/popup.html"
  }
}
```

This manifest is a planning example. Add content_scripts entries or register scripts appropriately for each supported domain. Do not claim support for a platform until its adapter has been implemented and tested.

9. Scheduled daily notifications

Users should be able to enable or disable the daily report and choose a time in their local time zone. Example: 21:00. The notification should summarise activity across all enabled platforms.

Example notification

ScrollMeter — Your daily report
Today you watched 42 short videos across 3 platforms.
YouTube Shorts: 18 • Instagram Reels: 14 • TikTok: 10
Unique creators: 16
Estimated watch time: 31 minutes
Open ScrollMeter to view details.

Scheduling logic

• Save `dailyReportEnabled` and `dailyReportTime` in local settings.

• Use `chrome.alarms` to schedule the next local-time occurrence.

• When the alarm fires, aggregate today's events and create a `chrome.notifications` notification.

• Schedule the next day's alarm after sending the report.

• Recalculate the alarm after a settings change and when the extension starts.

Important limitations: the browser must be running for extension alarms to be processed; sleep, shutdown, browser settings, or OS notification permissions can delay or suppress notifications. Do not promise delivery at an exact minute in every condition.

```
// Illustrative scheduling approach
async function scheduleDailyReport(timeString) {
  await chrome.alarms.clear("daily-summary");

  const [hour, minute] = timeString.split(":").map(Number);
  const now = new Date();
  const next = new Date();
  next.setHours(hour, minute, 0, 0);

  if (next <= now) next.setDate(next.getDate() + 1);

  chrome.alarms.create("daily-summary", {
    when: next.getTime()
  });
}

chrome.alarms.onAlarm.addListener(async alarm => {
  if (alarm.name !== "daily-summary") return;

  const summary = await buildTodaySummary();
  await chrome.notifications.create("daily-summary", {
    type: "basic",
    iconUrl: "icons/icon128.png",
    title: "Your ScrollMeter daily report",
    message: `You watched ${summary.total} short videos today.`,
    priority: 1
  });

  await scheduleDailyReport(await getSavedReportTime());
});
```

The snippet is illustrative. Implement and test `buildTodaySummary`, settings validation, alarm restoration, notification permission handling, and a real icon before release.

10. Privacy and data protection

• Default to local-only storage. Do not upload browsing activity to a server in the MVP.

• Do not collect passwords, private messages, private group content, contact lists, or unrelated browsing history.

• Provide clear pause, delete, export, and per-platform controls.

• Explain what page data is inspected and why, in plain language.

• If cloud sync or analytics is introduced, make it opt-in, minimise data, secure it, and update the privacy policy and store disclosures.

• Use data retention settings so users can delete old history automatically.

This is product and engineering guidance, not legal advice. Before publishing, review current Chrome Web Store policies, relevant privacy laws, and the terms/policies of every supported platform.

11. Development roadmap

Phase	Scope	Definition of done
0 — Prototype	One platform only; event detection and local storage.	Can detect a visible short video, count it once, and show a basic total.
1 — MVP	Add dashboard, recent history, pause, clear, and creator grouping.	Reliable end-to-end flow on the first supported platform.
2 — Multi-platform	Add one adapter at a time for the other target platforms.	Each platform independently passes detection and regression tests.
3 — Notifications	Daily report settings and scheduled notifications.	User can select time, save it, and receive a report while browser is running.
4 — Analytics	Weekly trends, platform comparison, creator ranking, export.	Statistics match underlying event records and explain estimates.
5 — Release	Privacy policy, store listing, accessibility, support, QA.	Store policy review complete and release package tested.

Recommended order: do not attempt all platforms at once. Build the shared event model and prove the detection approach on one platform first. Then add adapters individually. This avoids shipping a beautiful dashboard with unreliable data.

12. Acceptance criteria

• A user can install the extension in a supported desktop browser.

• A user can enable or disable tracking separately for each supported platform.

• A qualifying short-form video creates no more than one event per configured de-duplication window.

• The dashboard shows today's total and a per-platform breakdown.

• Creator counts are grouped only when creator identity is available; unknown creators are not guessed.

• Recent activity shows the platform and a timestamp; links are shown only when safely available.

• A user can pause tracking, clear local history, and export their data.

• A user can choose a daily notification time, disable notifications, and change the schedule.

• The notification summary is based on stored events and labels estimated watch time appropriately.

• The extension remains usable if one platform adapter fails or the platform changes its layout.

13. Testing and maintenance

• Test signed-in and signed-out states where relevant, without attempting to bypass access controls.

• Test feed scrolling, autoplay, paused videos, rapid scrolling, preloaded videos, and browser tab changes.

• Test duplicate prevention across route changes and page re-renders.

• Test missing creator names, unavailable post URLs, and platform layout changes.

• Test browser restart, computer sleep/wake, notification permission denial, and changed notification times.

• Keep platform adapters isolated so a site update does not break the entire extension.

• Maintain a small manual regression checklist for each supported platform after every release.

14. Deliverables expected from a developer

Deliverable	Expected output
Working extension	Installable Chrome/Edge extension package with source code.
Platform adapters	Separate and documented adapters for each platform actually supported.
Dashboard UI	Minimal popup with overview, platform totals, creators, and recent activity.
Notification feature	User-configurable daily report time, enable/disable control, and notification summary.
Privacy controls	Pause, per-platform controls, clear history, export, and retention behaviour.
Documentation	Setup instructions, architecture notes, supported-platform list, limitations, and maintenance notes.
QA report	Tested browser versions, platforms/routes, known limitations, and outstanding bugs.

15. Final product direction

Build a trustworthy usage-awareness tool, not a surveillance tool. The product should help users understand their own short-video consumption across supported social websites. Keep tracking transparent, local-first, easy to pause, and honest about detection accuracy.

The key differentiator is a single, simple view of short-form video consumption across multiple platforms, with creator-level insights and a daily reminder chosen by the user.

Before implementation begins, confirm the first-release platform list, browser support, whether the user wants an in-page overlay, and the retention/privacy defaults. These choices determine permissions, testing scope, and development effort.